

Branch and bound

Michel Bierlaire

Algorithm

```
[1]: import numpy as np
```

We illustrate here the branch and bound algorithm on the traveling salesman problem. Note that we do not rely on mathematical modeling. We simply exploit the structure of the problem to calculate bounds.

This is Algorithm 26.1 in Bierlaire (2015) Optimization: principles and algorithms, EPFL Press.

1 Traveling salesman problem

The following class implements the symmetric traveling salesman problem. It takes as input the distance between every pair of two cities. In particular, it allows to calculate tour lengths, lower bounds, and tour extension. These are the problem specific ingredients that are needed by the branch and bound algorithm.

1.1 Lower bound calculation

The calculation of the lower bound can be done using the following reasoning. Each city in the tour involves two different arcs, one to reach the city, and one to leave it. A lower bound on the length of the two arcs incident to a city is obtained by considering the two shortest arcs incident to that city. If we compute this lower bound for all nodes and add them up, we have double counted each arc (once for each of the two incident cities). Therefore, a lower bound on the total path is obtained by dividing this quantity by two.

Note that the method described in this document is designed to illustrate the branch and bound algorithm. These lower bounds are quite poor, and this is not an efficient way to solve the problem.

```
[2]: class TSP:
    """
    Data management
    """
    def __init__(self, home, distances):
        self.distances = distances
        self.cities = set([c for k in self.distances.keys() for c in k])
        self.home = home
        if home not in self.cities:
```

```

        raise Exception(f'Home ({home}) does not belong to the list of
→cities: {self.cities}')
    def distance(self, c1, c2):
        if (c1, c2) in self.distances:
            return self.distances[(c1, c2)]
        elif (c2, c1) in self.distances:
            return self.distances[(c2, c1)]
        else:
            raise Exception(f'Unknown pair of cities: {c1}-{c2}')

    def _contributionToLowerBound(self, city, fixed):
        if len(fixed) > 2:
            raise Exception(f'A maximum of two cities can be fixed: {fixed}')
        if city in fixed:
            raise Exception(f'City {city} cannot be in the list of fixed cities:
→{fixed}')

        # First calculate the distance from city to all other cities, excluding
→those that are fixed.
        allValues = [self.distance(city, i) for i in self.cities - set([city]) -
→set(fixed)]
        allValues.sort()
        if fixed:
            # The contributions to the lower bound combines fixed cities, and
            values = [self.distance(city, i) for i in fixed] + allValues[:
→2-len(fixed)]
        else:
            values = allValues[:2]
        return sum(values)

    def tourLength(self, tour):
        if len(tour) != len(self.cities):
            raise Exception(f'This is not a valid tour. It covers {len(tour)}
→cities and not {len(self.cities)}')
        pairs = [(tour[i], tour[i + 1]) for i in range(len(tour) - 1)] +
→[(tour[-1], tour[0])]
        d = [self.distance(c1, c2) for (c1, c2) in pairs]
        return sum(d)

    def lowerBound(self, partialTour):
        # If all cities are involved, the tour is complete and the lower bound
→is the length of the tour.
        if len(partialTour) == len(self.cities):
            return self.tourLength(partialTour)

```

```

        fixedPairs = [(city, partialTour[i + 1]) for i, city in
→enumerate(partialTour[:-1])]
        fixedCities = {c: list() for c in self.cities}
        for c1, c2 in fixedPairs:
            fixedCities[c1].append(c2)
            fixedCities[c2].append(c1)
        bounds = [self._contributionToLowerBound(c, fixedCities[c]) for c in
→self.cities]
        return sum(bounds) / 2.0

    def extendTour(self, partialTour):
        if len(partialTour) == len(self.cities):
            raise Exception(f'Tour is already complete, Cannot be extended:
→{partialTour}')
        nonVisitedCities = self.cities - set(partialTour)
        newTours = [partialTour + [c] for c in nonVisitedCities]
        return newTours

```

2 Subproblem

The branch and bound algorithm maintains a list of subproblems. The following class implements a subproblem. The key function is the “solve” function. It solves the problem to optimality if it is easy to do so, either because the problem itself is easy (typically, because only one city needs to be inserted), or because all the subproblems have been solved or discarded.

```

[3]: class subproblem:
    def __init__(self, tsp, partialTour):
        self.tsp = tsp
        self.partialTour = partialTour
        self.branches = None
        self.lowerBound = self.tsp.lowerBound(self.partialTour)
        self.optimalValue = None
        self.optimalTour = None
        self.subproblems = None

    def __str__(self):
        if self.optimalValue is not None:
            return f'{self.partialTour} (Opt: {self.optimalValue})'
        elif self.lowerBound is not None:
            return f'{self.partialTour} [LB: {self.lowerBound}]'
        else:
            return f'{self.partialTour}'

    def isFeasible(self):
        # For TSP, all subproblems are feasible

```

```

return True

def calculateLowerBound(self):
    self.lowerBound = self.tsp.lowerBound(self.partialTour)
    return self.lowerBound

def solve(self):
    # This function solves the problem is it is easy to do so, and returns
    →False otherwise.
    # It also creates the branches, that is the subproblems.

    # If all the cities have been included, except one, the problem is
    →trivial to solve.
    if len(self.partialTour) == len(self.tsp.cities) - 1:
        extendedTour = self.tsp.extendTour(self.partialTour)
        self.optimalTour = extendedTour[0]
        self.optimalValue = self.tsp.tourLength(self.optimalTour)
        return True

    # If no subproblem has been created, we create them
    if self.subproblems is None:
        extendedTours = self.tsp.extendTour(self.partialTour)
        self.subproblems = [subproblem(self.tsp, pt) for pt in extendedTours]
        return False

    bestTour = None
    bestValue = np.inf
    # We loop on the subproblems. If they have all been solved, the best
    →solution
    # is the optimal solution of the current subproblem.
    # If the optimal value is np.inf, it means that the problem has been
    →discarded.
    for p in self.subproblems:
        if p.optimalValue is None:
            # The problem has not been treated. No way to solve the
            →subproblem.
            return False
        else:
            if p.optimalValue < bestValue:
                bestValue = p.optimalValue
                bestTour = p.optimalTour
    self.optimalValue = bestValue
    self.optimalTour = bestTour
    return True

```

3 Branch & bound

This is the branch and bound algorithm for the symmetric TSP problem.

```
[4]: def branchAndBound(tsp, firstTour=None):
    if firstTour is None:
        currentBestTour = [tsp.home] + [c for c in set(tsp.cities) - set([tsp.
→home])]
    else:
        currentBestTour = firstTour
    upperBound = tsp.tourLength(currentBestTour)
    fullProblem = subproblem(tsp, [tsp.home])
    setOfProblems = [fullProblem]
    iter = 0
    while setOfProblems:
        iter += 1
        currentProblem = setOfProblems[-1]
        if currentProblem.optimalValue == np.inf:
            setOfProblems.pop()
        else:
            print(f'==== Iteration {iter} =====')
            print(f'Current best tour: {currentBestTour} Length: {upperBound}')
            print(f'Treat problem {currentProblem}')
            lb = currentProblem.calculateLowerBound()
            if currentProblem.solve():
                if currentProblem.optimalValue < upperBound:
                    currentBestTour = currentProblem.optimalTour
                    upperBound = currentProblem.optimalValue
                    # Remove the subproblem from the list
                    print(f'--> Optimal solution of {currentProblem}:_
→{currentProblem.optimalTour}')
                    setOfProblems.pop()
                else:
                    print(f'--> Branch')
                    setOfProblems += currentProblem.subproblems
            msg = [f'\t{p.__str__()} ' for p in setOfProblems if p.optimalValue !
→= np.inf]
            if len(msg) == 0:
                print('No more problem to treat.')
            else:
                print('List of problems to treat:')
                print('\n'.join(msg))

            # Discard problems with a lower bound
            # worse than the current upperbound
            for p in setOfProblems:
                if p.optimalValue != np.inf and p.lowerBound >= upperBound:
```

```

        print(f'Discard {p} because lb={p.lowerBound} >=_{
→{upperBound}}')
        p.optimalValue = np.inf
    return fullProblem

```

4 Example 1 - 4 cities

```

[5]: distances = {('L', 'G'): 64,
                  ('L', 'B'): 104,
                  ('L', 'Z'): 228,
                  ('G', 'B'): 158,
                  ('G', 'Z'): 279,
                  ('B', 'Z'): 125}
tsp = TSP('L', distances)

```

```

[6]: fullProblem = branchAndBound(tsp,['L', 'Z', 'B', 'G'])

```

```

===== Iteration 1 =====
Current best tour: ['L', 'Z', 'B', 'G'] Length: 575
Treat problem ['L'] [LB: 486.0]
--> Branch
List of problems to treat:
    ['L'] [LB: 486.0]
    ['L', 'B'] [LB: 486.0]
    ['L', 'G'] [LB: 486.0]
    ['L', 'Z'] [LB: 548.0]
===== Iteration 2 =====
Current best tour: ['L', 'Z', 'B', 'G'] Length: 575
Treat problem ['L', 'Z'] [LB: 548.0]
--> Branch
List of problems to treat:
    ['L'] [LB: 486.0]
    ['L', 'B'] [LB: 486.0]
    ['L', 'G'] [LB: 486.0]
    ['L', 'Z'] [LB: 548.0]
    ['L', 'Z', 'B'] [LB: 548.0]
    ['L', 'Z', 'G'] [LB: 685.5]
Discard ['L', 'Z', 'G'] [LB: 685.5] because lb=685.5 >= 575
===== Iteration 4 =====
Current best tour: ['L', 'Z', 'B', 'G'] Length: 575
Treat problem ['L', 'Z', 'B'] [LB: 548.0]
--> Optimal solution of ['L', 'Z', 'B'] (Opt: 575): ['L', 'Z', 'B', 'G']
List of problems to treat:
    ['L'] [LB: 486.0]
    ['L', 'B'] [LB: 486.0]
    ['L', 'G'] [LB: 486.0]

```

```

        ['L', 'Z'] [LB: 548.0]
===== Iteration 5 =====
Current best tour: ['L', 'Z', 'B', 'G'] Length: 575
Treat problem ['L', 'Z'] [LB: 548.0]
--> Optimal solution of ['L', 'Z'] (Opt: 575): ['L', 'Z', 'B', 'G']
List of problems to treat:
        ['L'] [LB: 486.0]
        ['L', 'B'] [LB: 486.0]
        ['L', 'G'] [LB: 486.0]
===== Iteration 6 =====
Current best tour: ['L', 'Z', 'B', 'G'] Length: 575
Treat problem ['L', 'G'] [LB: 486.0]
--> Branch
List of problems to treat:
        ['L'] [LB: 486.0]
        ['L', 'B'] [LB: 486.0]
        ['L', 'G'] [LB: 486.0]
        ['L', 'G', 'B'] [LB: 502.5]
        ['L', 'G', 'Z'] [LB: 572.0]
===== Iteration 7 =====
Current best tour: ['L', 'Z', 'B', 'G'] Length: 575
Treat problem ['L', 'G', 'Z'] [LB: 572.0]
--> Optimal solution of ['L', 'G', 'Z'] (Opt: 572): ['L', 'G', 'Z', 'B']
List of problems to treat:
        ['L'] [LB: 486.0]
        ['L', 'B'] [LB: 486.0]
        ['L', 'G'] [LB: 486.0]
        ['L', 'G', 'B'] [LB: 502.5]
===== Iteration 8 =====
Current best tour: ['L', 'G', 'Z', 'B'] Length: 572
Treat problem ['L', 'G', 'B'] [LB: 502.5]
--> Optimal solution of ['L', 'G', 'B'] (Opt: 575): ['L', 'G', 'B', 'Z']
List of problems to treat:
        ['L'] [LB: 486.0]
        ['L', 'B'] [LB: 486.0]
        ['L', 'G'] [LB: 486.0]
===== Iteration 9 =====
Current best tour: ['L', 'G', 'Z', 'B'] Length: 572
Treat problem ['L', 'G'] [LB: 486.0]
--> Optimal solution of ['L', 'G'] (Opt: 572): ['L', 'G', 'Z', 'B']
List of problems to treat:
        ['L'] [LB: 486.0]
        ['L', 'B'] [LB: 486.0]
===== Iteration 10 =====
Current best tour: ['L', 'G', 'Z', 'B'] Length: 572
Treat problem ['L', 'B'] [LB: 486.0]
--> Branch
List of problems to treat:

```

```

['L'] [LB: 486.0]
['L', 'B'] [LB: 486.0]
['L', 'B', 'G'] [LB: 502.5]
['L', 'B', 'Z'] [LB: 486.0]
===== Iteration 11 =====
Current best tour: ['L', 'G', 'Z', 'B'] Length: 572
Treat problem ['L', 'B', 'Z'] [LB: 486.0]
--> Optimal solution of ['L', 'B', 'Z'] (Opt: 572): ['L', 'B', 'Z', 'G']
List of problems to treat:
    ['L'] [LB: 486.0]
    ['L', 'B'] [LB: 486.0]
    ['L', 'B', 'G'] [LB: 502.5]
===== Iteration 12 =====
Current best tour: ['L', 'G', 'Z', 'B'] Length: 572
Treat problem ['L', 'B', 'G'] [LB: 502.5]
--> Optimal solution of ['L', 'B', 'G'] (Opt: 769): ['L', 'B', 'G', 'Z']
List of problems to treat:
    ['L'] [LB: 486.0]
    ['L', 'B'] [LB: 486.0]
===== Iteration 13 =====
Current best tour: ['L', 'G', 'Z', 'B'] Length: 572
Treat problem ['L', 'B'] [LB: 486.0]
--> Optimal solution of ['L', 'B'] (Opt: 572): ['L', 'B', 'Z', 'G']
List of problems to treat:
    ['L'] [LB: 486.0]
===== Iteration 14 =====
Current best tour: ['L', 'G', 'Z', 'B'] Length: 572
Treat problem ['L'] [LB: 486.0]
--> Optimal solution of ['L'] (Opt: 572): ['L', 'B', 'Z', 'G']
No more problem to treat.

```

```

[7]: print(f'Optimal tour: {fullProblem.optimalTour}')
     print(f'Length of the optimal tour: {fullProblem.optimalValue}')

```

```

Optimal tour: ['L', 'B', 'Z', 'G']
Length of the optimal tour: 572

```

5 Example 2 - 4 cities

```

[8]: distances = {('1', '2'): 10,
                  ('1', '3'): 15,
                  ('1', '4'): 20,
                  ('2', '3'): 35,
                  ('2', '4'): 25,
                  ('3', '4'): 30}
     tsp = TSP('1', distances)

```



```
[9]: fullProblem = branchAndBound(tsp)
```

```
===== Iteration 1 =====
Current best tour: ['1', '2', '3', '4'] Length: 95
Treat problem ['1'] [LB: 75.0]
--> Branch
List of problems to treat:
    ['1'] [LB: 75.0]
    ['1', '2'] [LB: 75.0]
    ['1', '3'] [LB: 75.0]
    ['1', '4'] [LB: 77.5]
===== Iteration 2 =====
Current best tour: ['1', '2', '3', '4'] Length: 95
Treat problem ['1', '4'] [LB: 77.5]
--> Branch
List of problems to treat:
    ['1'] [LB: 75.0]
    ['1', '2'] [LB: 75.0]
    ['1', '3'] [LB: 75.0]
    ['1', '4'] [LB: 77.5]
    ['1', '4', '2'] [LB: 77.5]
    ['1', '4', '3'] [LB: 80.0]
===== Iteration 3 =====
Current best tour: ['1', '2', '3', '4'] Length: 95
Treat problem ['1', '4', '3'] [LB: 80.0]
--> Optimal solution of ['1', '4', '3'] (Opt: 95): ['1', '4', '3', '2']
List of problems to treat:
    ['1'] [LB: 75.0]
    ['1', '2'] [LB: 75.0]
    ['1', '3'] [LB: 75.0]
    ['1', '4'] [LB: 77.5]
    ['1', '4', '2'] [LB: 77.5]
===== Iteration 4 =====
Current best tour: ['1', '2', '3', '4'] Length: 95
Treat problem ['1', '4', '2'] [LB: 77.5]
--> Optimal solution of ['1', '4', '2'] (Opt: 95): ['1', '4', '2', '3']
List of problems to treat:
    ['1'] [LB: 75.0]
    ['1', '2'] [LB: 75.0]
    ['1', '3'] [LB: 75.0]
    ['1', '4'] [LB: 77.5]
===== Iteration 5 =====
Current best tour: ['1', '2', '3', '4'] Length: 95
Treat problem ['1', '4'] [LB: 77.5]
--> Optimal solution of ['1', '4'] (Opt: 95): ['1', '4', '2', '3']
List of problems to treat:
    ['1'] [LB: 75.0]
```

```

        ['1', '2'] [LB: 75.0]
        ['1', '3'] [LB: 75.0]
===== Iteration 6 =====
Current best tour: ['1', '2', '3', '4'] Length: 95
Treat problem ['1', '3'] [LB: 75.0]
--> Branch
List of problems to treat:
        ['1'] [LB: 75.0]
        ['1', '2'] [LB: 75.0]
        ['1', '3'] [LB: 75.0]
        ['1', '3', '2'] [LB: 82.5]
        ['1', '3', '4'] [LB: 77.5]
===== Iteration 7 =====
Current best tour: ['1', '2', '3', '4'] Length: 95
Treat problem ['1', '3', '4'] [LB: 77.5]
--> Optimal solution of ['1', '3', '4'] (Opt: 80): ['1', '3', '4', '2']
List of problems to treat:
        ['1'] [LB: 75.0]
        ['1', '2'] [LB: 75.0]
        ['1', '3'] [LB: 75.0]
        ['1', '3', '2'] [LB: 82.5]
Discard ['1', '3', '2'] [LB: 82.5] because lb=82.5 >= 80
===== Iteration 9 =====
Current best tour: ['1', '3', '4', '2'] Length: 80
Treat problem ['1', '3'] [LB: 75.0]
--> Optimal solution of ['1', '3'] (Opt: 80): ['1', '3', '4', '2']
List of problems to treat:
        ['1'] [LB: 75.0]
        ['1', '2'] [LB: 75.0]
===== Iteration 10 =====
Current best tour: ['1', '3', '4', '2'] Length: 80
Treat problem ['1', '2'] [LB: 75.0]
--> Branch
List of problems to treat:
        ['1'] [LB: 75.0]
        ['1', '2'] [LB: 75.0]
        ['1', '2', '4'] [LB: 75.0]
        ['1', '2', '3'] [LB: 82.5]
Discard ['1', '2', '3'] [LB: 82.5] because lb=82.5 >= 80
===== Iteration 12 =====
Current best tour: ['1', '3', '4', '2'] Length: 80
Treat problem ['1', '2', '4'] [LB: 75.0]
--> Optimal solution of ['1', '2', '4'] (Opt: 80): ['1', '2', '4', '3']
List of problems to treat:
        ['1'] [LB: 75.0]
        ['1', '2'] [LB: 75.0]
===== Iteration 13 =====
Current best tour: ['1', '3', '4', '2'] Length: 80

```

```
Treat problem ['1', '2'] [LB: 75.0]
--> Optimal solution of ['1', '2'] (Opt: 80): ['1', '2', '4', '3']
List of problems to treat:
    ['1'] [LB: 75.0]
===== Iteration 14 =====
Current best tour: ['1', '3', '4', '2'] Length: 80
Treat problem ['1'] [LB: 75.0]
--> Optimal solution of ['1'] (Opt: 80): ['1', '2', '4', '3']
No more problem to treat.
```

```
[10]: print(f'Optimal tour: {fullProblem.optimalTour}')
      print(f'Length of the optimal tour: {fullProblem.optimalValue}')
```

```
Optimal tour: ['1', '2', '4', '3']
Length of the optimal tour: 80
```

6 Example 3 - 5 cities

```
[11]: distances = {('A', 'B'): 100,
                  ('A', 'C'): 300,
                  ('A', 'D'): 100,
                  ('A', 'E'): 75,
                  ('B', 'C'): 50,
                  ('B', 'D'): 75,
                  ('B', 'E'): 125,
                  ('C', 'D'): 100,
                  ('C', 'E'): 125,
                  ('D', 'E'): 50}
      tsp = TSP('E', distances)
```

```
[12]: fullProblem = branchAndBound(tsp)
```

```
===== Iteration 1 =====
Current best tour: ['E', 'B', 'A', 'C', 'D'] Length: 675
Treat problem ['E'] [LB: 350.0]
--> Branch
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'B'] [LB: 400.0]
    ['E', 'A'] [LB: 350.0]
    ['E', 'C'] [LB: 387.5]
    ['E', 'D'] [LB: 350.0]
===== Iteration 2 =====
Current best tour: ['E', 'B', 'A', 'C', 'D'] Length: 675
Treat problem ['E', 'D'] [LB: 350.0]
--> Branch
List of problems to treat:
```

```

['E'] [LB: 350.0]
['E', 'B'] [LB: 400.0]
['E', 'A'] [LB: 350.0]
['E', 'C'] [LB: 387.5]
['E', 'D'] [LB: 350.0]
['E', 'D', 'B'] [LB: 350.0]
['E', 'D', 'A'] [LB: 362.5]
['E', 'D', 'C'] [LB: 362.5]
===== Iteration 3 =====
Current best tour: ['E', 'B', 'A', 'C', 'D'] Length: 675
Treat problem ['E', 'D', 'C'] [LB: 362.5]
--> Branch
List of problems to treat:
['E'] [LB: 350.0]
['E', 'B'] [LB: 400.0]
['E', 'A'] [LB: 350.0]
['E', 'C'] [LB: 387.5]
['E', 'D'] [LB: 350.0]
['E', 'D', 'B'] [LB: 350.0]
['E', 'D', 'A'] [LB: 362.5]
['E', 'D', 'C'] [LB: 362.5]
['E', 'D', 'C', 'B'] [LB: 362.5]
['E', 'D', 'C', 'A'] [LB: 587.5]
===== Iteration 4 =====
Current best tour: ['E', 'B', 'A', 'C', 'D'] Length: 675
Treat problem ['E', 'D', 'C', 'A'] [LB: 587.5]
--> Optimal solution of ['E', 'D', 'C', 'A'] (Opt: 675): ['E', 'D', 'C', 'A',
'B']
List of problems to treat:
['E'] [LB: 350.0]
['E', 'B'] [LB: 400.0]
['E', 'A'] [LB: 350.0]
['E', 'C'] [LB: 387.5]
['E', 'D'] [LB: 350.0]
['E', 'D', 'B'] [LB: 350.0]
['E', 'D', 'A'] [LB: 362.5]
['E', 'D', 'C'] [LB: 362.5]
['E', 'D', 'C', 'B'] [LB: 362.5]
===== Iteration 5 =====
Current best tour: ['E', 'B', 'A', 'C', 'D'] Length: 675
Treat problem ['E', 'D', 'C', 'B'] [LB: 362.5]
--> Optimal solution of ['E', 'D', 'C', 'B'] (Opt: 375): ['E', 'D', 'C', 'B',
'A']
List of problems to treat:
['E'] [LB: 350.0]
['E', 'B'] [LB: 400.0]
['E', 'A'] [LB: 350.0]
['E', 'C'] [LB: 387.5]

```

```

        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]
        ['E', 'D', 'A'] [LB: 362.5]
        ['E', 'D', 'C'] [LB: 362.5]
Discard ['E', 'B'] [LB: 400.0] because lb=400.0 >= 375
Discard ['E', 'C'] [LB: 387.5] because lb=387.5 >= 375
===== Iteration 6 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'D', 'C'] [LB: 362.5]
--> Optimal solution of ['E', 'D', 'C'] (Opt: 375): ['E', 'D', 'C', 'B', 'A']
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]
        ['E', 'D', 'A'] [LB: 362.5]
===== Iteration 7 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'D', 'A'] [LB: 362.5]
--> Branch
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]
        ['E', 'D', 'A'] [LB: 362.5]
        ['E', 'D', 'A', 'B'] [LB: 387.5]
        ['E', 'D', 'A', 'C'] [LB: 575.0]
Discard ['E', 'D', 'A', 'B'] [LB: 387.5] because lb=387.5 >= 375
Discard ['E', 'D', 'A', 'C'] [LB: 575.0] because lb=575.0 >= 375
===== Iteration 10 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'D', 'A'] [LB: 362.5]
--> Optimal solution of ['E', 'D', 'A'] (Opt: inf): None
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]
===== Iteration 11 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'D', 'B'] [LB: 350.0]
--> Branch
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]

```

```

        ['E', 'D', 'B', 'A'] [LB: 375.0]
        ['E', 'D', 'B', 'C'] [LB: 350.0]
Discard ['E', 'D', 'B', 'A'] [LB: 375.0] because lb=375.0 >= 375
===== Iteration 12 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'D', 'B', 'C'] [LB: 350.0]
--> Optimal solution of ['E', 'D', 'B', 'C'] (Opt: 550): ['E', 'D', 'B', 'C',
'A']
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]
===== Iteration 14 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'D', 'B'] [LB: 350.0]
--> Optimal solution of ['E', 'D', 'B'] (Opt: 550): ['E', 'D', 'B', 'C', 'A']
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
===== Iteration 15 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'D'] [LB: 350.0]
--> Optimal solution of ['E', 'D'] (Opt: 375): ['E', 'D', 'C', 'B', 'A']
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
===== Iteration 17 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'A'] [LB: 350.0]
--> Branch
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'A', 'B'] [LB: 362.5]
        ['E', 'A', 'C'] [LB: 550.0]
        ['E', 'A', 'D'] [LB: 362.5]
Discard ['E', 'A', 'C'] [LB: 550.0] because lb=550.0 >= 375
===== Iteration 18 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'A', 'D'] [LB: 362.5]
--> Branch
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'A', 'B'] [LB: 362.5]
        ['E', 'A', 'D'] [LB: 362.5]

```

```

        ['E', 'A', 'D', 'B'] [LB: 375.0]
        ['E', 'A', 'D', 'C'] [LB: 387.5]
Discard ['E', 'A', 'D', 'B'] [LB: 375.0] because lb=375.0 >= 375
Discard ['E', 'A', 'D', 'C'] [LB: 387.5] because lb=387.5 >= 375
===== Iteration 21 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'A', 'D'] [LB: 362.5]
--> Optimal solution of ['E', 'A', 'D'] (Opt: inf): None
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'A', 'B'] [LB: 362.5]
===== Iteration 23 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'A', 'B'] [LB: 362.5]
--> Branch
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'A', 'B'] [LB: 362.5]
        ['E', 'A', 'B', 'C'] [LB: 362.5]
        ['E', 'A', 'B', 'D'] [LB: 375.0]
Discard ['E', 'A', 'B', 'D'] [LB: 375.0] because lb=375.0 >= 375
===== Iteration 25 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'A', 'B', 'C'] [LB: 362.5]
--> Optimal solution of ['E', 'A', 'B', 'C'] (Opt: 375): ['E', 'A', 'B', 'C', 'D']
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'A', 'B'] [LB: 362.5]
===== Iteration 26 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'A', 'B'] [LB: 362.5]
--> Optimal solution of ['E', 'A', 'B'] (Opt: 375): ['E', 'A', 'B', 'C', 'D']
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
===== Iteration 27 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E', 'A'] [LB: 350.0]
--> Optimal solution of ['E', 'A'] (Opt: 375): ['E', 'A', 'B', 'C', 'D']
List of problems to treat:
        ['E'] [LB: 350.0]
===== Iteration 29 =====
Current best tour: ['E', 'D', 'C', 'B', 'A'] Length: 375
Treat problem ['E'] [LB: 350.0]

```

--> Optimal solution of ['E'] (Opt: 375): ['E', 'A', 'B', 'C', 'D']
No more problem to treat.

```
[13]: print(f'Optimal tour: {fullProblem.optimalTour}')  
      print(f'Length of the optimal tour: {fullProblem.optimalValue}')
```

Optimal tour: ['E', 'A', 'B', 'C', 'D']
Length of the optimal tour: 375

We run the algorithm again, starting with the optimal solution. We observe that it may take a while for the algorithm to realize that it is indeed the optimal solution.

```
[14]: fullProblem = branchAndBound(tsp,['E', 'A', 'B', 'C', 'D'])
```

```
===== Iteration 1 =====  
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375  
Treat problem ['E'] [LB: 350.0]  
--> Branch  
List of problems to treat:  
    ['E'] [LB: 350.0]  
    ['E', 'B'] [LB: 400.0]  
    ['E', 'A'] [LB: 350.0]  
    ['E', 'C'] [LB: 387.5]  
    ['E', 'D'] [LB: 350.0]  
Discard ['E', 'B'] [LB: 400.0] because lb=400.0 >= 375  
Discard ['E', 'C'] [LB: 387.5] because lb=387.5 >= 375  
===== Iteration 2 =====  
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375  
Treat problem ['E', 'D'] [LB: 350.0]  
--> Branch  
List of problems to treat:  
    ['E'] [LB: 350.0]  
    ['E', 'A'] [LB: 350.0]  
    ['E', 'D'] [LB: 350.0]  
    ['E', 'D', 'B'] [LB: 350.0]  
    ['E', 'D', 'A'] [LB: 362.5]  
    ['E', 'D', 'C'] [LB: 362.5]  
===== Iteration 3 =====  
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375  
Treat problem ['E', 'D', 'C'] [LB: 362.5]  
--> Branch  
List of problems to treat:  
    ['E'] [LB: 350.0]  
    ['E', 'A'] [LB: 350.0]  
    ['E', 'D'] [LB: 350.0]  
    ['E', 'D', 'B'] [LB: 350.0]  
    ['E', 'D', 'A'] [LB: 362.5]  
    ['E', 'D', 'C'] [LB: 362.5]
```



```

        ['E', 'D', 'C', 'B'] [LB: 362.5]
        ['E', 'D', 'C', 'A'] [LB: 587.5]
Discard ['E', 'D', 'C', 'A'] [LB: 587.5] because lb=587.5 >= 375
===== Iteration 5 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'D', 'C', 'B'] [LB: 362.5]
--> Optimal solution of ['E', 'D', 'C', 'B'] (Opt: 375): ['E', 'D', 'C', 'B',
'A']
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]
        ['E', 'D', 'A'] [LB: 362.5]
        ['E', 'D', 'C'] [LB: 362.5]
===== Iteration 6 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'D', 'C'] [LB: 362.5]
--> Optimal solution of ['E', 'D', 'C'] (Opt: 375): ['E', 'D', 'C', 'B', 'A']
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]
        ['E', 'D', 'A'] [LB: 362.5]
===== Iteration 7 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'D', 'A'] [LB: 362.5]
--> Branch
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]
        ['E', 'D', 'A'] [LB: 362.5]
        ['E', 'D', 'A', 'B'] [LB: 387.5]
        ['E', 'D', 'A', 'C'] [LB: 575.0]
Discard ['E', 'D', 'A', 'B'] [LB: 387.5] because lb=387.5 >= 375
Discard ['E', 'D', 'A', 'C'] [LB: 575.0] because lb=575.0 >= 375
===== Iteration 10 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'D', 'A'] [LB: 362.5]
--> Optimal solution of ['E', 'D', 'A'] (Opt: inf): None
List of problems to treat:
        ['E'] [LB: 350.0]
        ['E', 'A'] [LB: 350.0]
        ['E', 'D'] [LB: 350.0]
        ['E', 'D', 'B'] [LB: 350.0]

```

```

===== Iteration 11 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'D', 'B'] [LB: 350.0]
--> Branch
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]
    ['E', 'D'] [LB: 350.0]
    ['E', 'D', 'B'] [LB: 350.0]
    ['E', 'D', 'B', 'A'] [LB: 375.0]
    ['E', 'D', 'B', 'C'] [LB: 350.0]
Discard ['E', 'D', 'B', 'A'] [LB: 375.0] because lb=375.0 >= 375
===== Iteration 12 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'D', 'B', 'C'] [LB: 350.0]
--> Optimal solution of ['E', 'D', 'B', 'C'] (Opt: 550): ['E', 'D', 'B', 'C',
'A']
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]
    ['E', 'D'] [LB: 350.0]
    ['E', 'D', 'B'] [LB: 350.0]
===== Iteration 14 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'D', 'B'] [LB: 350.0]
--> Optimal solution of ['E', 'D', 'B'] (Opt: 550): ['E', 'D', 'B', 'C', 'A']
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]
    ['E', 'D'] [LB: 350.0]
===== Iteration 15 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'D'] [LB: 350.0]
--> Optimal solution of ['E', 'D'] (Opt: 375): ['E', 'D', 'C', 'B', 'A']
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]
===== Iteration 17 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'A'] [LB: 350.0]
--> Branch
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]
    ['E', 'A', 'B'] [LB: 362.5]
    ['E', 'A', 'C'] [LB: 550.0]
    ['E', 'A', 'D'] [LB: 362.5]
Discard ['E', 'A', 'C'] [LB: 550.0] because lb=550.0 >= 375

```

```

===== Iteration 18 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'A', 'D'] [LB: 362.5]
--> Branch
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]
    ['E', 'A', 'B'] [LB: 362.5]
    ['E', 'A', 'D'] [LB: 362.5]
    ['E', 'A', 'D', 'B'] [LB: 375.0]
    ['E', 'A', 'D', 'C'] [LB: 387.5]
Discard ['E', 'A', 'D', 'B'] [LB: 375.0] because lb=375.0 >= 375
Discard ['E', 'A', 'D', 'C'] [LB: 387.5] because lb=387.5 >= 375
===== Iteration 21 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'A', 'D'] [LB: 362.5]
--> Optimal solution of ['E', 'A', 'D'] (Opt: inf): None
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]
    ['E', 'A', 'B'] [LB: 362.5]
===== Iteration 23 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'A', 'B'] [LB: 362.5]
--> Branch
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]
    ['E', 'A', 'B'] [LB: 362.5]
    ['E', 'A', 'B', 'C'] [LB: 362.5]
    ['E', 'A', 'B', 'D'] [LB: 375.0]
Discard ['E', 'A', 'B', 'D'] [LB: 375.0] because lb=375.0 >= 375
===== Iteration 25 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'A', 'B', 'C'] [LB: 362.5]
--> Optimal solution of ['E', 'A', 'B', 'C'] (Opt: 375): ['E', 'A', 'B', 'C',
'D']
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]
    ['E', 'A', 'B'] [LB: 362.5]
===== Iteration 26 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'A', 'B'] [LB: 362.5]
--> Optimal solution of ['E', 'A', 'B'] (Opt: 375): ['E', 'A', 'B', 'C', 'D']
List of problems to treat:
    ['E'] [LB: 350.0]
    ['E', 'A'] [LB: 350.0]

```

```

===== Iteration 27 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E', 'A'] [LB: 350.0]
--> Optimal solution of ['E', 'A'] (Opt: 375): ['E', 'A', 'B', 'C', 'D']
List of problems to treat:
    ['E'] [LB: 350.0]
===== Iteration 29 =====
Current best tour: ['E', 'A', 'B', 'C', 'D'] Length: 375
Treat problem ['E'] [LB: 350.0]
--> Optimal solution of ['E'] (Opt: 375): ['E', 'A', 'B', 'C', 'D']
No more problem to treat.

```

```

[15]: print(f'Optimal tour: {fullProblem.optimalTour}')
      print(f'Length of the optimal tour: {fullProblem.optimalValue}')

```

```

Optimal tour: ['E', 'A', 'B', 'C', 'D']
Length of the optimal tour: 375

```

7 Random example

We considered n cities whose coordinates are randomly generated.

```

[16]: import random as rnd
      n = 6
      x = [rnd.uniform(0, 10) for i in range(n)]
      y = [rnd.uniform(0, 10) for i in range(n)]
      names = [f'{i}' for i in range(1, n + 1)]

```

```

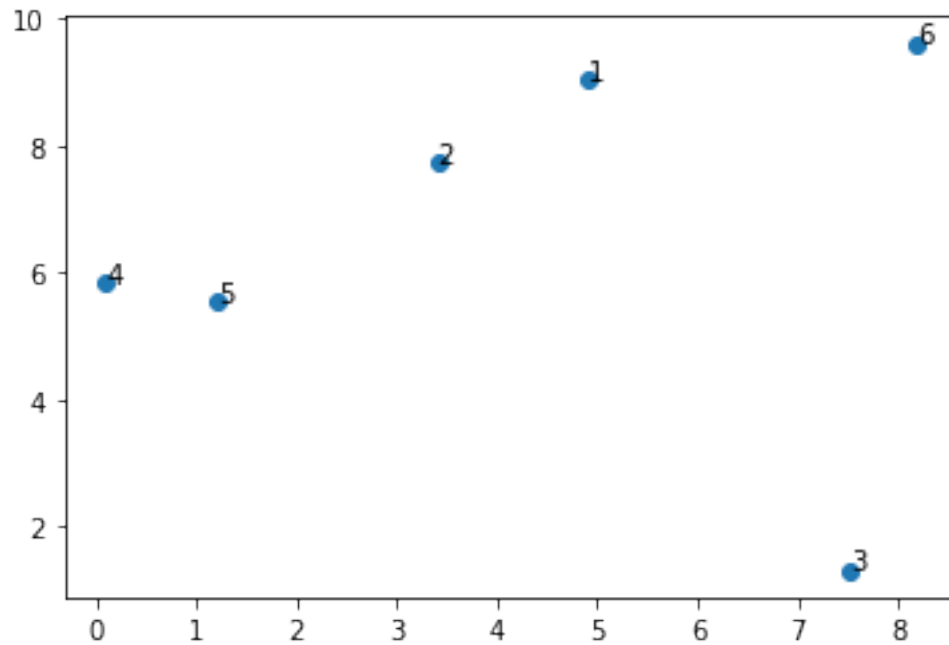
[17]: import matplotlib.pyplot as plt

```

```

[18]: fig, ax = plt.subplots()
      ax.scatter(x, y)
      for i in range(n):
          ax.annotate(names[i], (x[i], y[i]))

```



And we consider the Euclidean distance between them.

```
[19]: distances = {(names[i], names[j]):
    np.sqrt((x[i] - x[j]) * (x[i] - x[j]) +
            (y[i] - y[j]) * (y[i] - y[j]))
    for i in range(n) for j in range(i + 1, n) }
tsp = TSP('1', distances)
```

```
[20]: fullProblem = branchAndBound(tsp)
```

=====
Iteration 1
=====

Current best tour: ['1', '3', '6', '5', '2', '4'] Length: 20.38987784245154

Treat problem ['1'] [LB: 11.622630641882171]

--> Branch

List of problems to treat:

```
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
```

=====
Iteration 2
=====

Current best tour: ['1', '3', '6', '5', '2', '4'] Length: 20.38987784245154

Treat problem ['1', '4'] [LB: 11.622630641882171]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]

===== Iteration 3 =====

Current best tour: ['1', '3', '6', '5', '2', '4'] Length: 20.38987784245154

Treat problem ['1', '4', '6'] [LB: 11.732720505967407]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '3'] [LB: 13.703973611817371]
['1', '4', '6', '5'] [LB: 14.478161732003876]

===== Iteration 4 =====

Current best tour: ['1', '3', '6', '5', '2', '4'] Length: 20.38987784245154

Treat problem ['1', '4', '6', '5'] [LB: 14.478161732003876]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '3'] [LB: 13.703973611817371]
['1', '4', '6', '5'] [LB: 14.478161732003876]
['1', '4', '6', '5', '2'] [LB: 16.956240144856203]
['1', '4', '6', '5', '3'] [LB: 15.617298103354909]

===== Iteration 5 =====

Current best tour: ['1', '3', '6', '5', '2', '4'] Length: 20.38987784245154

Treat problem ['1', '4', '6', '5', '3'] [LB: 15.617298103354909]

--> Optimal solution of ['1', '4', '6', '5', '3'] (Opt: 18.86411576283066):
['1', '4', '6', '5', '3', '2']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '3'] [LB: 13.703973611817371]
['1', '4', '6', '5'] [LB: 14.478161732003876]
['1', '4', '6', '5', '2'] [LB: 16.956240144856203]

===== Iteration 6 =====

Current best tour: ['1', '4', '6', '5', '3', '2'] Length: 18.86411576283066

Treat problem ['1', '4', '6', '5', '2'] [LB: 16.956240144856203]

--> Optimal solution of ['1', '4', '6', '5', '2'] (Opt: 20.84482264740161):
['1', '4', '6', '5', '2', '3']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '3'] [LB: 13.703973611817371]
['1', '4', '6', '5'] [LB: 14.478161732003876]

===== Iteration 7 =====

Current best tour: ['1', '4', '6', '5', '3', '2'] Length: 18.86411576283066

Treat problem ['1', '4', '6', '5'] [LB: 14.478161732003876]

--> Optimal solution of ['1', '4', '6', '5'] (Opt: 18.86411576283066): ['1',
'4', '6', '5', '3', '2']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]

```

['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '3'] [LB: 13.703973611817371]
===== Iteration 8 =====
Current best tour: ['1', '4', '6', '5', '3', '2'] Length: 18.86411576283066
Treat problem ['1', '4', '6', '3'] [LB: 13.703973611817371]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '3'] [LB: 13.703973611817371]
['1', '4', '6', '3', '2'] [LB: 15.870194414669674]
['1', '4', '6', '3', '5'] [LB: 14.684245249359606]
===== Iteration 9 =====
Current best tour: ['1', '4', '6', '5', '3', '2'] Length: 18.86411576283066
Treat problem ['1', '4', '6', '3', '5'] [LB: 14.684245249359606]
--> Optimal solution of ['1', '4', '6', '3', '5'] (Opt: 18.242920518835383):
['1', '4', '6', '3', '5', '2']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '3'] [LB: 13.703973611817371]
['1', '4', '6', '3', '2'] [LB: 15.870194414669674]
===== Iteration 10 =====

```


Current best tour: ['1', '4', '6', '3', '5', '2'] Length: 18.242920518835383
Treat problem ['1', '4', '6', '3', '2'] [LB: 15.870194414669674]
--> Optimal solution of ['1', '4', '6', '3', '2'] (Opt: 18.229205473490467):
['1', '4', '6', '3', '2', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '3'] [LB: 13.703973611817371]

=====
Iteration 11
=====

Current best tour: ['1', '4', '6', '3', '2', '5'] Length: 18.229205473490467
Treat problem ['1', '4', '6', '3'] [LB: 13.703973611817371]
--> Optimal solution of ['1', '4', '6', '3'] (Opt: 18.229205473490467): ['1',
'4', '6', '3', '2', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]

=====
Iteration 12
=====

Current best tour: ['1', '4', '6', '3', '2', '5'] Length: 18.229205473490467
Treat problem ['1', '4', '6', '2'] [LB: 11.732720505967407]
--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]

```

['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '2', '3'] [LB: 13.555668845219708]
['1', '4', '6', '2', '5'] [LB: 13.708661721410936]
===== Iteration 13 =====
Current best tour: ['1', '4', '6', '3', '2', '5'] Length: 18.229205473490467
Treat problem ['1', '4', '6', '2', '5'] [LB: 13.708661721410936]
--> Optimal solution of ['1', '4', '6', '2', '5'] (Opt: 16.53036241267781):
['1', '4', '6', '2', '5', '3']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
['1', '4', '6', '2', '3'] [LB: 13.555668845219708]
===== Iteration 14 =====
Current best tour: ['1', '4', '6', '2', '5', '3'] Length: 16.53036241267781
Treat problem ['1', '4', '6', '2', '3'] [LB: 13.555668845219708]
--> Optimal solution of ['1', '4', '6', '2', '3'] (Opt: 14.535940482761944):
['1', '4', '6', '2', '3', '5']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
['1', '4', '6', '2'] [LB: 11.732720505967407]
===== Iteration 15 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '6', '2'] [LB: 11.732720505967407]
--> Optimal solution of ['1', '4', '6', '2'] (Opt: 14.535940482761944): ['1',
'4', '6', '2', '3', '5']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]

```

```

['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '6'] [LB: 11.732720505967407]
===== Iteration 16 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '6'] [LB: 11.732720505967407]
--> Optimal solution of ['1', '4', '6'] (Opt: 14.535940482761944): ['1', '4',
'6', '2', '3', '5']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
===== Iteration 17 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '5'] [LB: 12.363432169784659]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '5', '2'] [LB: 14.841510582636985]
['1', '4', '5', '3'] [LB: 13.502568541135691]
['1', '4', '5', '6'] [LB: 15.11785088576743]
Discard ['1', '4', '5', '2'] [LB: 14.841510582636985] because
lb=14.841510582636985 >= 14.535940482761944
Discard ['1', '4', '5', '6'] [LB: 15.11785088576743] because
lb=15.11785088576743 >= 14.535940482761944
===== Iteration 19 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '5', '3'] [LB: 13.502568541135691]
--> Branch
List of problems to treat:

```

```

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
['1', '4', '5', '3'] [LB: 13.502568541135691]
['1', '4', '5', '3', '2'] [LB: 15.668789343987994]
['1', '4', '5', '3', '6'] [LB: 15.32393440312316]
Discard ['1', '4', '5', '3', '2'] [LB: 15.668789343987994] because
lb=15.668789343987994 >= 14.535940482761944
Discard ['1', '4', '5', '3', '6'] [LB: 15.32393440312316] because
lb=15.32393440312316 >= 14.535940482761944
===== Iteration 22 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '5', '3'] [LB: 13.502568541135691]
--> Optimal solution of ['1', '4', '5', '3'] (Opt: inf): None
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '5'] [LB: 12.363432169784659]
===== Iteration 24 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '5'] [LB: 12.363432169784659]
--> Optimal solution of ['1', '4', '5'] (Opt: inf): None
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
===== Iteration 25 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '3'] [LB: 12.657158880126893]
--> Branch
List of problems to treat:

```

```

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '3', '2'] [LB: 14.480107219379194]
['1', '4', '3', '5'] [LB: 13.294158054069126]
['1', '4', '3', '6'] [LB: 14.135252278514361]

```

===== Iteration 26 =====

Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944

Treat problem ['1', '4', '3', '6'] [LB: 14.135252278514361]

--> Branch

List of problems to treat:

```

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '3', '2'] [LB: 14.480107219379194]
['1', '4', '3', '5'] [LB: 13.294158054069126]
['1', '4', '3', '6'] [LB: 14.135252278514361]
['1', '4', '3', '6', '2'] [LB: 14.135252278514361]
['1', '4', '3', '6', '5'] [LB: 16.88069350455083]

```

Discard ['1', '4', '3', '6', '5'] [LB: 16.88069350455083] because

lb=16.88069350455083 >= 14.535940482761944

===== Iteration 28 =====

Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944

Treat problem ['1', '4', '3', '6', '2'] [LB: 14.135252278514361]

--> Optimal solution of ['1', '4', '3', '6', '2'] (Opt: 16.111193493957888):

['1', '4', '3', '6', '2', '5']

List of problems to treat:

```

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '3', '2'] [LB: 14.480107219379194]
['1', '4', '3', '5'] [LB: 13.294158054069126]
['1', '4', '3', '6'] [LB: 14.135252278514361]

```

===== Iteration 29 =====

Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944

Treat problem ['1', '4', '3', '6'] [LB: 14.135252278514361]

--> Optimal solution of ['1', '4', '3', '6'] (Opt: 16.111193493957888): ['1', '4', '3', '6', '2', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '3', '2'] [LB: 14.480107219379194]
['1', '4', '3', '5'] [LB: 13.294158054069126]

===== Iteration 30 =====

Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944

Treat problem ['1', '4', '3', '5'] [LB: 13.294158054069126]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '3'] [LB: 12.657158880126893]
['1', '4', '3', '2'] [LB: 14.480107219379194]
['1', '4', '3', '5'] [LB: 13.294158054069126]
['1', '4', '3', '5', '2'] [LB: 15.772236466921454]
['1', '4', '3', '5', '6'] [LB: 16.0485767700519]

Discard ['1', '4', '3', '5', '2'] [LB: 15.772236466921454] because

lb=15.772236466921454 >= 14.535940482761944

Discard ['1', '4', '3', '5', '6'] [LB: 16.0485767700519] because

lb=16.0485767700519 >= 14.535940482761944

===== Iteration 33 =====

Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944

Treat problem ['1', '4', '3', '5'] [LB: 13.294158054069126]

--> Optimal solution of ['1', '4', '3', '5'] (Opt: inf): None

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]

```

        ['1', '4', '2'] [LB: 11.622630641882171]
        ['1', '4', '3'] [LB: 12.657158880126893]
        ['1', '4', '3', '2'] [LB: 14.480107219379194]
===== Iteration 34 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '3', '2'] [LB: 14.480107219379194]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '4'] [LB: 11.622630641882171]
    ['1', '4', '2'] [LB: 11.622630641882171]
    ['1', '4', '3'] [LB: 12.657158880126893]
    ['1', '4', '3', '2'] [LB: 14.480107219379194]
    ['1', '4', '3', '2', '5'] [LB: 16.839118278199983]
    ['1', '4', '3', '2', '6'] [LB: 14.480107219379194]
Discard ['1', '4', '3', '2', '5'] [LB: 16.839118278199983] because
lb=16.839118278199983 >= 14.535940482761944
===== Iteration 35 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '3', '2', '6'] [LB: 14.480107219379194]
--> Optimal solution of ['1', '4', '3', '2', '6'] (Opt: 16.73238873795317):
['1', '4', '3', '2', '6', '5']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '4'] [LB: 11.622630641882171]
    ['1', '4', '2'] [LB: 11.622630641882171]
    ['1', '4', '3'] [LB: 12.657158880126893]
    ['1', '4', '3', '2'] [LB: 14.480107219379194]
===== Iteration 37 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '3', '2'] [LB: 14.480107219379194]
--> Optimal solution of ['1', '4', '3', '2'] (Opt: 16.73238873795317): ['1',
'4', '3', '2', '6', '5']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '4'] [LB: 11.622630641882171]

```

```

        ['1', '4', '2'] [LB: 11.622630641882171]
        ['1', '4', '3'] [LB: 12.657158880126893]
===== Iteration 38 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '3'] [LB: 12.657158880126893]
--> Optimal solution of ['1', '4', '3'] (Opt: 16.111193493957888): ['1', '4',
'3', '6', '2', '5']
List of problems to treat:
        ['1'] [LB: 11.622630641882171]
        ['1', '3'] [LB: 12.96192249819801]
        ['1', '6'] [LB: 12.714007869450782]
        ['1', '5'] [LB: 11.622630641882171]
        ['1', '2'] [LB: 12.320157655128359]
        ['1', '4'] [LB: 11.622630641882171]
        ['1', '4', '2'] [LB: 11.622630641882171]
===== Iteration 39 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '2'] [LB: 11.622630641882171]
--> Branch
List of problems to treat:
        ['1'] [LB: 11.622630641882171]
        ['1', '3'] [LB: 12.96192249819801]
        ['1', '6'] [LB: 12.714007869450782]
        ['1', '5'] [LB: 11.622630641882171]
        ['1', '2'] [LB: 12.320157655128359]
        ['1', '4'] [LB: 11.622630641882171]
        ['1', '4', '2'] [LB: 11.622630641882171]
        ['1', '4', '2', '3'] [LB: 13.828648824511738]
        ['1', '4', '2', '5'] [LB: 13.981641700702966]
        ['1', '4', '2', '6'] [LB: 11.622630641882171]
===== Iteration 40 =====
Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '2', '6'] [LB: 11.622630641882171]
--> Branch
List of problems to treat:
        ['1'] [LB: 11.622630641882171]
        ['1', '3'] [LB: 12.96192249819801]
        ['1', '6'] [LB: 12.714007869450782]
        ['1', '5'] [LB: 11.622630641882171]
        ['1', '2'] [LB: 12.320157655128359]
        ['1', '4'] [LB: 11.622630641882171]
        ['1', '4', '2'] [LB: 11.622630641882171]
        ['1', '4', '2', '3'] [LB: 13.828648824511738]
        ['1', '4', '2', '5'] [LB: 13.981641700702966]
        ['1', '4', '2', '6'] [LB: 11.622630641882171]
        ['1', '4', '2', '6', '3'] [LB: 13.10072404026964]
        ['1', '4', '2', '6', '5'] [LB: 13.874912160456145]
===== Iteration 41 =====

```


Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '2', '6', '5'] [LB: 13.874912160456145]
--> Optimal solution of ['1', '4', '2', '6', '5'] (Opt: 16.696612851723017):
['1', '4', '2', '6', '5', '3']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '2', '3'] [LB: 13.828648824511738]
['1', '4', '2', '5'] [LB: 13.981641700702966]
['1', '4', '2', '6'] [LB: 11.622630641882171]
['1', '4', '2', '6', '3'] [LB: 13.10072404026964]

=====
Iteration 42
=====

Current best tour: ['1', '4', '6', '2', '3', '5'] Length: 14.535940482761944
Treat problem ['1', '4', '2', '6', '3'] [LB: 13.10072404026964]
--> Optimal solution of ['1', '4', '2', '6', '3'] (Opt: 14.080995677811874):
['1', '4', '2', '6', '3', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '2', '3'] [LB: 13.828648824511738]
['1', '4', '2', '5'] [LB: 13.981641700702966]
['1', '4', '2', '6'] [LB: 11.622630641882171]

=====
Iteration 43
=====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '4', '2', '6'] [LB: 11.622630641882171]
--> Optimal solution of ['1', '4', '2', '6'] (Opt: 14.080995677811874): ['1',
'4', '2', '6', '3', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '4'] [LB: 11.622630641882171]
['1', '4', '2'] [LB: 11.622630641882171]
['1', '4', '2', '3'] [LB: 13.828648824511738]
['1', '4', '2', '5'] [LB: 13.981641700702966]

=====
Iteration 44
=====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '4', '2', '5'] [LB: 13.981641700702966]

--> Branch

List of problems to treat:

- ['1'] [LB: 11.622630641882171]
- ['1', '3'] [LB: 12.96192249819801]
- ['1', '6'] [LB: 12.714007869450782]
- ['1', '5'] [LB: 11.622630641882171]
- ['1', '2'] [LB: 12.320157655128359]
- ['1', '4'] [LB: 11.622630641882171]
- ['1', '4', '2'] [LB: 11.622630641882171]
- ['1', '4', '2', '3'] [LB: 13.828648824511738]
- ['1', '4', '2', '5'] [LB: 13.981641700702966]
- ['1', '4', '2', '5', '3'] [LB: 15.120778072053998]
- ['1', '4', '2', '5', '6'] [LB: 16.736060416685735]

Discard ['1', '4', '2', '5', '3'] [LB: 15.120778072053998] because
lb=15.120778072053998 >= 14.080995677811874

Discard ['1', '4', '2', '5', '6'] [LB: 16.736060416685735] because
lb=16.736060416685735 >= 14.080995677811874

=====
Iteration 47
=====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '4', '2', '5'] [LB: 13.981641700702966]

--> Optimal solution of ['1', '4', '2', '5'] (Opt: inf): None

List of problems to treat:

- ['1'] [LB: 11.622630641882171]
- ['1', '3'] [LB: 12.96192249819801]
- ['1', '6'] [LB: 12.714007869450782]
- ['1', '5'] [LB: 11.622630641882171]
- ['1', '2'] [LB: 12.320157655128359]
- ['1', '4'] [LB: 11.622630641882171]
- ['1', '4', '2'] [LB: 11.622630641882171]
- ['1', '4', '2', '3'] [LB: 13.828648824511738]

=====
Iteration 48
=====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '4', '2', '3'] [LB: 13.828648824511738]

--> Branch

List of problems to treat:

- ['1'] [LB: 11.622630641882171]
- ['1', '3'] [LB: 12.96192249819801]
- ['1', '6'] [LB: 12.714007869450782]
- ['1', '5'] [LB: 11.622630641882171]
- ['1', '2'] [LB: 12.320157655128359]
- ['1', '4'] [LB: 11.622630641882171]
- ['1', '4', '2'] [LB: 11.622630641882171]
- ['1', '4', '2', '3'] [LB: 13.828648824511738]
- ['1', '4', '2', '3', '5'] [LB: 14.808920462053972]
- ['1', '4', '2', '3', '6'] [LB: 15.650014686499205]

Discard ['1', '4', '2', '3', '5'] [LB: 14.808920462053972] because

```

lb=14.808920462053972 >= 14.080995677811874
Discard ['1', '4', '2', '3', '6'] [LB: 15.650014686499205] because
lb=15.650014686499205 >= 14.080995677811874
===== Iteration 51 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '4', '2', '3'] [LB: 13.828648824511738]
--> Optimal solution of ['1', '4', '2', '3'] (Opt: inf): None
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '4'] [LB: 11.622630641882171]
    ['1', '4', '2'] [LB: 11.622630641882171]
===== Iteration 52 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '4', '2'] [LB: 11.622630641882171]
--> Optimal solution of ['1', '4', '2'] (Opt: 14.080995677811874): ['1', '4',
'2', '6', '3', '5']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '4'] [LB: 11.622630641882171]
===== Iteration 53 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '4'] [LB: 11.622630641882171]
--> Optimal solution of ['1', '4'] (Opt: 14.080995677811874): ['1', '4', '2',
'6', '3', '5']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
===== Iteration 54 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2'] [LB: 12.320157655128359]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]

```

```

        ['1', '2', '4'] [LB: 12.703227498505619]
        ['1', '2', '3'] [LB: 14.526175837757926]
        ['1', '2', '5'] [LB: 14.67916871394915]
        ['1', '2', '6'] [LB: 12.320157655128359]
Discard ['1', '2', '3'] [LB: 14.526175837757926] because lb=14.526175837757926
>= 14.080995677811874
Discard ['1', '2', '5'] [LB: 14.67916871394915] because lb=14.67916871394915 >=
14.080995677811874
===== Iteration 55 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '6'] [LB: 12.320157655128359]
--> Branch
List of problems to treat:
        ['1'] [LB: 11.622630641882171]
        ['1', '3'] [LB: 12.96192249819801]
        ['1', '6'] [LB: 12.714007869450782]
        ['1', '5'] [LB: 11.622630641882171]
        ['1', '2'] [LB: 12.320157655128359]
        ['1', '2', '4'] [LB: 12.703227498505619]
        ['1', '2', '6'] [LB: 12.320157655128359]
        ['1', '2', '6', '4'] [LB: 12.393543412235122]
        ['1', '2', '6', '3'] [LB: 13.798251053515827]
        ['1', '2', '6', '5'] [LB: 14.572439173702332]
Discard ['1', '2', '6', '5'] [LB: 14.572439173702332] because
lb=14.572439173702332 >= 14.080995677811874
===== Iteration 57 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '6', '3'] [LB: 13.798251053515827]
--> Branch
List of problems to treat:
        ['1'] [LB: 11.622630641882171]
        ['1', '3'] [LB: 12.96192249819801]
        ['1', '6'] [LB: 12.714007869450782]
        ['1', '5'] [LB: 11.622630641882171]
        ['1', '2'] [LB: 12.320157655128359]
        ['1', '2', '4'] [LB: 12.703227498505619]
        ['1', '2', '6'] [LB: 12.320157655128359]
        ['1', '2', '6', '4'] [LB: 12.393543412235122]
        ['1', '2', '6', '3'] [LB: 13.798251053515827]
        ['1', '2', '6', '3', '4'] [LB: 14.796075184782076]
        ['1', '2', '6', '3', '5'] [LB: 14.778522691058061]
Discard ['1', '2', '6', '3', '4'] [LB: 14.796075184782076] because
lb=14.796075184782076 >= 14.080995677811874
Discard ['1', '2', '6', '3', '5'] [LB: 14.778522691058061] because
lb=14.778522691058061 >= 14.080995677811874
===== Iteration 60 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '6', '3'] [LB: 13.798251053515827]

```

```

--> Optimal solution of ['1', '2', '6', '3'] (Opt: inf): None
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '2', '4'] [LB: 12.703227498505619]
    ['1', '2', '6'] [LB: 12.320157655128359]
    ['1', '2', '6', '4'] [LB: 12.393543412235122]
===== Iteration 61 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '6', '4'] [LB: 12.393543412235122]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '2', '4'] [LB: 12.703227498505619]
    ['1', '2', '6'] [LB: 12.320157655128359]
    ['1', '2', '6', '4'] [LB: 12.393543412235122]
    ['1', '2', '6', '4', '3'] [LB: 13.428071650479843]
    ['1', '2', '6', '4', '5'] [LB: 13.13434494013761]
===== Iteration 62 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '6', '4', '5'] [LB: 13.13434494013761]
--> Optimal solution of ['1', '2', '6', '4', '5'] (Opt: 16.1580058549197): ['1',
'2', '6', '4', '5', '3']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '2', '4'] [LB: 12.703227498505619]
    ['1', '2', '6'] [LB: 12.320157655128359]
    ['1', '2', '6', '4'] [LB: 12.393543412235122]
    ['1', '2', '6', '4', '3'] [LB: 13.428071650479843]
===== Iteration 63 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '6', '4', '3'] [LB: 13.428071650479843]
--> Optimal solution of ['1', '2', '6', '4', '3'] (Opt: 14.267031047937293):
['1', '2', '6', '4', '3', '5']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]

```

```

['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '2', '4'] [LB: 12.703227498505619]
['1', '2', '6'] [LB: 12.320157655128359]
['1', '2', '6', '4'] [LB: 12.393543412235122]
===== Iteration 64 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '6', '4'] [LB: 12.393543412235122]
--> Optimal solution of ['1', '2', '6', '4'] (Opt: 14.267031047937293): ['1',
'2', '6', '4', '3', '5']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '2', '4'] [LB: 12.703227498505619]
['1', '2', '6'] [LB: 12.320157655128359]
===== Iteration 65 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '6'] [LB: 12.320157655128359]
--> Optimal solution of ['1', '2', '6'] (Opt: 14.267031047937293): ['1', '2',
'6', '4', '3', '5']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '2', '4'] [LB: 12.703227498505619]
===== Iteration 68 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '4'] [LB: 12.703227498505619]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '2', '4'] [LB: 12.703227498505619]
['1', '2', '4', '3'] [LB: 13.70105162977187]
['1', '2', '4', '5'] [LB: 13.407324919429636]
['1', '2', '4', '6'] [LB: 12.776613255612384]
===== Iteration 69 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '4', '6'] [LB: 12.776613255612384]

```

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '2', '4'] [LB: 12.703227498505619]
['1', '2', '4', '3'] [LB: 13.70105162977187]
['1', '2', '4', '5'] [LB: 13.407324919429636]
['1', '2', '4', '6'] [LB: 12.776613255612384]
['1', '2', '4', '6', '3'] [LB: 14.747866361462348]
['1', '2', '4', '6', '5'] [LB: 15.522054481648857]

Discard ['1', '2', '4', '6', '3'] [LB: 14.747866361462348] because
lb=14.747866361462348 >= 14.080995677811874

Discard ['1', '2', '4', '6', '5'] [LB: 15.522054481648857] because
lb=15.522054481648857 >= 14.080995677811874

===== Iteration 72 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '2', '4', '6'] [LB: 12.776613255612384]

--> Optimal solution of ['1', '2', '4', '6'] (Opt: inf): None

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '2', '4'] [LB: 12.703227498505619]
['1', '2', '4', '3'] [LB: 13.70105162977187]
['1', '2', '4', '5'] [LB: 13.407324919429636]

===== Iteration 73 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '2', '4', '5'] [LB: 13.407324919429636]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
['1', '2', '4'] [LB: 12.703227498505619]
['1', '2', '4', '3'] [LB: 13.70105162977187]
['1', '2', '4', '5'] [LB: 13.407324919429636]
['1', '2', '4', '5', '3'] [LB: 14.54646129078067]
['1', '2', '4', '5', '6'] [LB: 16.161743635412407]

Discard ['1', '2', '4', '5', '3'] [LB: 14.54646129078067] because
lb=14.54646129078067 >= 14.080995677811874

Discard ['1', '2', '4', '5', '6'] [LB: 16.161743635412407] because

```

lb=16.161743635412407 >= 14.080995677811874
===== Iteration 76 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '4', '5'] [LB: 13.407324919429636]
--> Optimal solution of ['1', '2', '4', '5'] (Opt: inf): None
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '2', '4'] [LB: 12.703227498505619]
    ['1', '2', '4', '3'] [LB: 13.70105162977187]
===== Iteration 77 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '4', '3'] [LB: 13.70105162977187]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '2', '4'] [LB: 12.703227498505619]
    ['1', '2', '4', '3'] [LB: 13.70105162977187]
    ['1', '2', '4', '3', '5'] [LB: 14.338050803714106]
    ['1', '2', '4', '3', '6'] [LB: 15.179145028159338]
Discard ['1', '2', '4', '3', '5'] [LB: 14.338050803714106] because
lb=14.338050803714106 >= 14.080995677811874
Discard ['1', '2', '4', '3', '6'] [LB: 15.179145028159338] because
lb=15.179145028159338 >= 14.080995677811874
===== Iteration 80 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '4', '3'] [LB: 13.70105162977187]
--> Optimal solution of ['1', '2', '4', '3'] (Opt: inf): None
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '2'] [LB: 12.320157655128359]
    ['1', '2', '4'] [LB: 12.703227498505619]
===== Iteration 81 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2', '4'] [LB: 12.703227498505619]
--> Optimal solution of ['1', '2', '4'] (Opt: inf): None
List of problems to treat:
    ['1'] [LB: 11.622630641882171]

```



```

['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '2'] [LB: 12.320157655128359]
===== Iteration 82 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '2'] [LB: 12.320157655128359]
--> Optimal solution of ['1', '2'] (Opt: 14.267031047937293): ['1', '2', '6',
'4', '3', '5']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
===== Iteration 83 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5'] [LB: 11.622630641882171]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '6'] [LB: 13.874912160456145]
===== Iteration 84 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '6'] [LB: 13.874912160456145]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '6'] [LB: 13.874912160456145]
['1', '5', '6', '2'] [LB: 13.874912160456145]
['1', '5', '6', '3'] [LB: 15.846165266306109]
['1', '5', '6', '4'] [LB: 14.441457625025405]
Discard ['1', '5', '6', '3'] [LB: 15.846165266306109] because
lb=15.846165266306109 >= 14.080995677811874
Discard ['1', '5', '6', '4'] [LB: 14.441457625025405] because
lb=14.441457625025405 >= 14.080995677811874
===== Iteration 87 =====

```

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '6', '2'] [LB: 13.874912160456145]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '6'] [LB: 13.874912160456145]
['1', '5', '6', '2'] [LB: 13.874912160456145]
['1', '5', '6', '2', '4'] [LB: 13.874912160456145]
['1', '5', '6', '2', '3'] [LB: 15.697860499708447]

Discard ['1', '5', '6', '2', '3'] [LB: 15.697860499708447] because
lb=15.697860499708447 >= 14.080995677811874

===== Iteration 89 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '5', '6', '2', '4'] [LB: 13.874912160456145]

--> Optimal solution of ['1', '5', '6', '2', '4'] (Opt: 16.41398837155345):

['1', '5', '6', '2', '4', '3']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '6'] [LB: 13.874912160456145]
['1', '5', '6', '2'] [LB: 13.874912160456145]

===== Iteration 90 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '5', '6', '2'] [LB: 13.874912160456145]

--> Optimal solution of ['1', '5', '6', '2'] (Opt: 16.41398837155345): ['1',
'5', '6', '2', '4', '3']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '6'] [LB: 13.874912160456145]

===== Iteration 91 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

```

Treat problem ['1', '5', '6'] [LB: 13.874912160456145]
--> Optimal solution of ['1', '5', '6'] (Opt: 16.41398837155345): ['1', '5',
'6', '2', '4', '3']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '5', '2'] [LB: 13.598571857325702]
    ['1', '5', '3'] [LB: 12.259629815824404]
    ['1', '5', '4'] [LB: 12.326728062806188]
===== Iteration 92 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '4'] [LB: 12.326728062806188]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '5', '2'] [LB: 13.598571857325702]
    ['1', '5', '3'] [LB: 12.259629815824404]
    ['1', '5', '4'] [LB: 12.326728062806188]
    ['1', '5', '4', '2'] [LB: 12.326728062806188]
    ['1', '5', '4', '3'] [LB: 13.361256301050908]
    ['1', '5', '4', '6'] [LB: 12.436817926891422]
===== Iteration 93 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '4', '6'] [LB: 12.436817926891422]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '5', '2'] [LB: 13.598571857325702]
    ['1', '5', '3'] [LB: 12.259629815824404]
    ['1', '5', '4'] [LB: 12.326728062806188]
    ['1', '5', '4', '2'] [LB: 12.326728062806188]
    ['1', '5', '4', '3'] [LB: 13.361256301050908]
    ['1', '5', '4', '6'] [LB: 12.436817926891422]
    ['1', '5', '4', '6', '2'] [LB: 12.436817926891422]
    ['1', '5', '4', '6', '3'] [LB: 14.408071032741386]
Discard ['1', '5', '4', '6', '3'] [LB: 14.408071032741386] because
lb=14.408071032741386 >= 14.080995677811874
===== Iteration 95 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '4', '6', '2'] [LB: 12.436817926891422]

```

--> Optimal solution of ['1', '5', '4', '6', '2'] (Opt: 16.144290809574784):
['1', '5', '4', '6', '2', '3']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '4', '2'] [LB: 12.326728062806188]
['1', '5', '4', '3'] [LB: 13.361256301050908]
['1', '5', '4', '6'] [LB: 12.436817926891422]

===== Iteration 96 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '5', '4', '6'] [LB: 12.436817926891422]

--> Optimal solution of ['1', '5', '4', '6'] (Opt: 16.144290809574784): ['1',
'5', '4', '6', '2', '3']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '4', '2'] [LB: 12.326728062806188]
['1', '5', '4', '3'] [LB: 13.361256301050908]

===== Iteration 97 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '5', '4', '3'] [LB: 13.361256301050908]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '4', '2'] [LB: 12.326728062806188]
['1', '5', '4', '3'] [LB: 13.361256301050908]
['1', '5', '4', '3', '2'] [LB: 15.184204640303209]
['1', '5', '4', '3', '6'] [LB: 14.839349699438376]

Discard ['1', '5', '4', '3', '2'] [LB: 15.184204640303209] because
lb=15.184204640303209 >= 14.080995677811874

Discard ['1', '5', '4', '3', '6'] [LB: 14.839349699438376] because
lb=14.839349699438376 >= 14.080995677811874

===== Iteration 100 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '5', '4', '3'] [LB: 13.361256301050908]

--> Optimal solution of ['1', '5', '4', '3'] (Opt: inf): None

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '4', '2'] [LB: 12.326728062806188]

===== Iteration 101 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '5', '4', '2'] [LB: 12.326728062806188]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '4', '2'] [LB: 12.326728062806188]
['1', '5', '4', '2', '3'] [LB: 14.532746245435753]
['1', '5', '4', '2', '6'] [LB: 12.326728062806188]

Discard ['1', '5', '4', '2', '3'] [LB: 14.532746245435753] because

lb=14.532746245435753 >= 14.080995677811874

===== Iteration 102 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '5', '4', '2', '6'] [LB: 12.326728062806188]

--> Optimal solution of ['1', '5', '4', '2', '6'] (Opt: 15.689346004624714):

['1', '5', '4', '2', '6', '3']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '4'] [LB: 12.326728062806188]
['1', '5', '4', '2'] [LB: 12.326728062806188]

===== Iteration 104 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1', '5', '4', '2'] [LB: 12.326728062806188]

--> Optimal solution of ['1', '5', '4', '2'] (Opt: 15.689346004624714): ['1',

```

'5', '4', '2', '6', '3']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '5', '2'] [LB: 13.598571857325702]
    ['1', '5', '3'] [LB: 12.259629815824404]
    ['1', '5', '4'] [LB: 12.326728062806188]
===== Iteration 105 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '4'] [LB: 12.326728062806188]
--> Optimal solution of ['1', '5', '4'] (Opt: 15.689346004624714): ['1', '5',
'4', '2', '6', '3']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '5', '2'] [LB: 13.598571857325702]
    ['1', '5', '3'] [LB: 12.259629815824404]
===== Iteration 106 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '3'] [LB: 12.259629815824404]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '5', '2'] [LB: 13.598571857325702]
    ['1', '5', '3'] [LB: 12.259629815824404]
    ['1', '5', '3', '2'] [LB: 14.425850618676709]
    ['1', '5', '3', '4'] [LB: 13.257453947090655]
    ['1', '5', '3', '6'] [LB: 14.080995677811874]
Discard ['1', '5', '3', '2'] [LB: 14.425850618676709] because
lb=14.425850618676709 >= 14.080995677811874
Discard ['1', '5', '3', '6'] [LB: 14.080995677811874] because
lb=14.080995677811874 >= 14.080995677811874
===== Iteration 108 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '3', '4'] [LB: 13.257453947090655]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]

```

```

['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '3', '4'] [LB: 13.257453947090655]
['1', '5', '3', '4', '2'] [LB: 13.257453947090655]
['1', '5', '3', '4', '6'] [LB: 13.367543811175889]
===== Iteration 109 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '3', '4', '6'] [LB: 13.367543811175889]
--> Optimal solution of ['1', '5', '3', '4', '6'] (Opt: 14.267031047937293):
['1', '5', '3', '4', '6', '2']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '3', '4'] [LB: 13.257453947090655]
['1', '5', '3', '4', '2'] [LB: 13.257453947090655]
===== Iteration 110 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '3', '4', '2'] [LB: 13.257453947090655]
--> Optimal solution of ['1', '5', '3', '4', '2'] (Opt: 14.550791398174482):
['1', '5', '3', '4', '2', '6']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
['1', '5', '3', '4'] [LB: 13.257453947090655]
===== Iteration 111 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '3', '4'] [LB: 13.257453947090655]
--> Optimal solution of ['1', '5', '3', '4'] (Opt: 14.267031047937293): ['1',
'5', '3', '4', '6', '2']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '3'] [LB: 12.259629815824404]
===== Iteration 113 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '3'] [LB: 12.259629815824404]
--> Optimal solution of ['1', '5', '3'] (Opt: 14.267031047937293): ['1', '5',

```

```

'3', '4', '6', '2']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '5', '2'] [LB: 13.598571857325702]
===== Iteration 114 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '2'] [LB: 13.598571857325702]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '5', '2'] [LB: 13.598571857325702]
    ['1', '5', '2', '4'] [LB: 13.981641700702966]
    ['1', '5', '2', '3'] [LB: 15.804590039955265]
    ['1', '5', '2', '6'] [LB: 13.598571857325702]
Discard ['1', '5', '2', '3'] [LB: 15.804590039955265] because
lb=15.804590039955265 >= 14.080995677811874
===== Iteration 115 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '2', '6'] [LB: 13.598571857325702]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]
    ['1', '5', '2'] [LB: 13.598571857325702]
    ['1', '5', '2', '4'] [LB: 13.981641700702966]
    ['1', '5', '2', '6'] [LB: 13.598571857325702]
    ['1', '5', '2', '6', '4'] [LB: 13.671957614432465]
    ['1', '5', '2', '6', '3'] [LB: 15.07666525571317]
Discard ['1', '5', '2', '6', '3'] [LB: 15.07666525571317] because
lb=15.07666525571317 >= 14.080995677811874
===== Iteration 117 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '2', '6', '4'] [LB: 13.671957614432465]
--> Optimal solution of ['1', '5', '2', '6', '4'] (Opt: 16.247737932508244):
['1', '5', '2', '6', '4', '3']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '5'] [LB: 11.622630641882171]

```



```

['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '2', '4'] [LB: 13.981641700702966]
['1', '5', '2', '6'] [LB: 13.598571857325702]
===== Iteration 118 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '2', '6'] [LB: 13.598571857325702]
--> Optimal solution of ['1', '5', '2', '6'] (Opt: 16.247737932508244): ['1',
'5', '2', '6', '4', '3']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '2', '4'] [LB: 13.981641700702966]
===== Iteration 120 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '2', '4'] [LB: 13.981641700702966]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '2', '4'] [LB: 13.981641700702966]
['1', '5', '2', '4', '3'] [LB: 14.979465831969215]
['1', '5', '2', '4', '6'] [LB: 14.055027457809729]
Discard ['1', '5', '2', '4', '3'] [LB: 14.979465831969215] because
lb=14.979465831969215 >= 14.080995677811874
===== Iteration 121 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '2', '4', '6'] [LB: 14.055027457809729]
--> Optimal solution of ['1', '5', '2', '4', '6'] (Opt: 17.910805107090752):
['1', '5', '2', '4', '6', '3']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
['1', '5', '2', '4'] [LB: 13.981641700702966]
===== Iteration 123 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '2', '4'] [LB: 13.981641700702966]
--> Optimal solution of ['1', '5', '2', '4'] (Opt: 17.910805107090752): ['1',
'5', '2', '4', '6', '3']
List of problems to treat:

```

```

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
['1', '5', '2'] [LB: 13.598571857325702]
===== Iteration 124 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5', '2'] [LB: 13.598571857325702]
--> Optimal solution of ['1', '5', '2'] (Opt: 16.247737932508244): ['1', '5',
'2', '6', '4', '3']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '5'] [LB: 11.622630641882171]
===== Iteration 125 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '5'] [LB: 11.622630641882171]
--> Optimal solution of ['1', '5'] (Opt: 14.267031047937293): ['1', '5', '3',
'4', '6', '2']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
===== Iteration 126 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6'] [LB: 12.714007869450782]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '6', '2'] [LB: 12.714007869450782]
['1', '6', '3'] [LB: 14.685260975300746]
['1', '6', '4'] [LB: 13.280553334020043]
['1', '6', '5'] [LB: 15.459449095487251]
Discard ['1', '6', '3'] [LB: 14.685260975300746] because lb=14.685260975300746
>= 14.080995677811874
Discard ['1', '6', '5'] [LB: 15.459449095487251] because lb=15.459449095487251
>= 14.080995677811874
===== Iteration 128 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '4'] [LB: 13.280553334020043]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]

```

```

['1', '6', '2'] [LB: 12.714007869450782]
['1', '6', '4'] [LB: 13.280553334020043]
['1', '6', '4', '2'] [LB: 13.280553334020043]
['1', '6', '4', '3'] [LB: 14.315081572264763]
['1', '6', '4', '5'] [LB: 14.021354861922529]
Discard ['1', '6', '4', '3'] [LB: 14.315081572264763] because
lb=14.315081572264763 >= 14.080995677811874
===== Iteration 129 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '4', '5'] [LB: 14.021354861922529]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '6', '2'] [LB: 12.714007869450782]
['1', '6', '4'] [LB: 13.280553334020043]
['1', '6', '4', '2'] [LB: 13.280553334020043]
['1', '6', '4', '5'] [LB: 14.021354861922529]
['1', '6', '4', '5', '2'] [LB: 16.49943327477486]
['1', '6', '4', '5', '3'] [LB: 15.160491233273564]
Discard ['1', '6', '4', '5', '2'] [LB: 16.49943327477486] because
lb=16.49943327477486 >= 14.080995677811874
Discard ['1', '6', '4', '5', '3'] [LB: 15.160491233273564] because
lb=15.160491233273564 >= 14.080995677811874
===== Iteration 132 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '4', '5'] [LB: 14.021354861922529]
--> Optimal solution of ['1', '6', '4', '5'] (Opt: inf): None
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '6', '2'] [LB: 12.714007869450782]
['1', '6', '4'] [LB: 13.280553334020043]
['1', '6', '4', '2'] [LB: 13.280553334020043]
===== Iteration 134 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '4', '2'] [LB: 13.280553334020043]
--> Branch
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '6', '2'] [LB: 12.714007869450782]
['1', '6', '4'] [LB: 13.280553334020043]
['1', '6', '4', '2'] [LB: 13.280553334020043]
['1', '6', '4', '2', '3'] [LB: 15.486571516649612]

```

```

        ['1', '6', '4', '2', '5'] [LB: 15.639564392840835]
Discard ['1', '6', '4', '2', '3'] [LB: 15.486571516649612] because
lb=15.486571516649612 >= 14.080995677811874
Discard ['1', '6', '4', '2', '5'] [LB: 15.639564392840835] because
lb=15.639564392840835 >= 14.080995677811874
===== Iteration 137 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '4', '2'] [LB: 13.280553334020043]
--> Optimal solution of ['1', '6', '4', '2'] (Opt: inf): None
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '6', '2'] [LB: 12.714007869450782]
    ['1', '6', '4'] [LB: 13.280553334020043]
===== Iteration 138 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '4'] [LB: 13.280553334020043]
--> Optimal solution of ['1', '6', '4'] (Opt: inf): None
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '6', '2'] [LB: 12.714007869450782]
===== Iteration 140 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '2'] [LB: 12.714007869450782]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]
    ['1', '6', '2'] [LB: 12.714007869450782]
    ['1', '6', '2', '4'] [LB: 12.714007869450782]
    ['1', '6', '2', '3'] [LB: 14.536956208703085]
    ['1', '6', '2', '5'] [LB: 14.689949084894312]
Discard ['1', '6', '2', '3'] [LB: 14.536956208703085] because
lb=14.536956208703085 >= 14.080995677811874
Discard ['1', '6', '2', '5'] [LB: 14.689949084894312] because
lb=14.689949084894312 >= 14.080995677811874
===== Iteration 143 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '2', '4'] [LB: 12.714007869450782]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '6'] [LB: 12.714007869450782]

```

```

['1', '6', '2'] [LB: 12.714007869450782]
['1', '6', '2', '4'] [LB: 12.714007869450782]
['1', '6', '2', '4', '3'] [LB: 13.711832000717033]
['1', '6', '2', '4', '5'] [LB: 13.418105290374799]
===== Iteration 144 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '2', '4', '5'] [LB: 13.418105290374799]
--> Optimal solution of ['1', '6', '2', '4', '5'] (Opt: 16.44176620515689):
['1', '6', '2', '4', '5', '3']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '6', '2'] [LB: 12.714007869450782]
['1', '6', '2', '4'] [LB: 12.714007869450782]
['1', '6', '2', '4', '3'] [LB: 13.711832000717033]
===== Iteration 145 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '2', '4', '3'] [LB: 13.711832000717033]
--> Optimal solution of ['1', '6', '2', '4', '3'] (Opt: 14.550791398174482):
['1', '6', '2', '4', '3', '5']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '6', '2'] [LB: 12.714007869450782]
['1', '6', '2', '4'] [LB: 12.714007869450782]
===== Iteration 146 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '2', '4'] [LB: 12.714007869450782]
--> Optimal solution of ['1', '6', '2', '4'] (Opt: 14.550791398174482): ['1',
'6', '2', '4', '3', '5']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
['1', '6', '2'] [LB: 12.714007869450782]
===== Iteration 147 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '6', '2'] [LB: 12.714007869450782]
--> Optimal solution of ['1', '6', '2'] (Opt: 14.550791398174482): ['1', '6',
'2', '4', '3', '5']
List of problems to treat:
['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '6'] [LB: 12.714007869450782]
===== Iteration 148 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

```

```

Treat problem ['1', '6'] [LB: 12.714007869450782]
--> Optimal solution of ['1', '6'] (Opt: 14.550791398174482): ['1', '6', '2',
'4', '3', '5']
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
===== Iteration 149 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3'] [LB: 12.96192249819801]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '3', '2'] [LB: 15.128143301050315]
    ['1', '3', '4'] [LB: 13.959746629464261]
    ['1', '3', '5'] [LB: 13.942194135740245]
    ['1', '3', '6'] [LB: 14.78328836018548]
Discard ['1', '3', '2'] [LB: 15.128143301050315] because lb=15.128143301050315
>= 14.080995677811874
Discard ['1', '3', '6'] [LB: 14.78328836018548] because lb=14.78328836018548 >=
14.080995677811874
===== Iteration 151 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '5'] [LB: 13.942194135740245]
--> Branch
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '3', '4'] [LB: 13.959746629464261]
    ['1', '3', '5'] [LB: 13.942194135740245]
    ['1', '3', '5', '2'] [LB: 16.420272548592575]
    ['1', '3', '5', '4'] [LB: 15.14842875407306]
    ['1', '3', '5', '6'] [LB: 16.696612851723017]
Discard ['1', '3', '5', '2'] [LB: 16.420272548592575] because
lb=16.420272548592575 >= 14.080995677811874
Discard ['1', '3', '5', '4'] [LB: 15.14842875407306] because
lb=15.14842875407306 >= 14.080995677811874
Discard ['1', '3', '5', '6'] [LB: 16.696612851723017] because
lb=16.696612851723017 >= 14.080995677811874
===== Iteration 155 =====
Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '5'] [LB: 13.942194135740245]
--> Optimal solution of ['1', '3', '5'] (Opt: inf): None
List of problems to treat:
    ['1'] [LB: 11.622630641882171]
    ['1', '3'] [LB: 12.96192249819801]
    ['1', '3', '4'] [LB: 13.959746629464261]
===== Iteration 156 =====

```

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '4'] [LB: 13.959746629464261]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '3', '4'] [LB: 13.959746629464261]
['1', '3', '4', '2'] [LB: 13.959746629464261]
['1', '3', '4', '5'] [LB: 14.700548157366747]
['1', '3', '4', '6'] [LB: 14.069836493549495]

Discard ['1', '3', '4', '5'] [LB: 14.700548157366747] because
lb=14.700548157366747 >= 14.080995677811874

===== Iteration 157 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '4', '6'] [LB: 14.069836493549495]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '3', '4'] [LB: 13.959746629464261]
['1', '3', '4', '2'] [LB: 13.959746629464261]
['1', '3', '4', '6'] [LB: 14.069836493549495]
['1', '3', '4', '6', '2'] [LB: 14.069836493549495]
['1', '3', '4', '6', '5'] [LB: 16.815277719585968]

Discard ['1', '3', '4', '6', '5'] [LB: 16.815277719585968] because
lb=16.815277719585968 >= 14.080995677811874

===== Iteration 159 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '4', '6', '2'] [LB: 14.069836493549495]

--> Optimal solution of ['1', '3', '4', '6', '2'] (Opt: 16.247737932508244):
['1', '3', '4', '6', '2', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '3', '4'] [LB: 13.959746629464261]
['1', '3', '4', '2'] [LB: 13.959746629464261]
['1', '3', '4', '6'] [LB: 14.069836493549495]

===== Iteration 160 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '4', '6'] [LB: 14.069836493549495]

--> Optimal solution of ['1', '3', '4', '6'] (Opt: 16.247737932508244): ['1',
'3', '4', '6', '2', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '3', '4'] [LB: 13.959746629464261]
['1', '3', '4', '2'] [LB: 13.959746629464261]

===== Iteration 162 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '4', '2'] [LB: 13.959746629464261]

--> Branch

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '3', '4'] [LB: 13.959746629464261]
['1', '3', '4', '2'] [LB: 13.959746629464261]
['1', '3', '4', '2', '5'] [LB: 16.318757688285054]
['1', '3', '4', '2', '6'] [LB: 13.959746629464261]

Discard ['1', '3', '4', '2', '5'] [LB: 16.318757688285054] because
lb=16.318757688285054 >= 14.080995677811874

===== Iteration 163 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '4', '2', '6'] [LB: 13.959746629464261]

--> Optimal solution of ['1', '3', '4', '2', '6'] (Opt: 16.41398837155345):
['1', '3', '4', '2', '6', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '3', '4'] [LB: 13.959746629464261]
['1', '3', '4', '2'] [LB: 13.959746629464261]

===== Iteration 165 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '4', '2'] [LB: 13.959746629464261]

--> Optimal solution of ['1', '3', '4', '2'] (Opt: 16.41398837155345): ['1',
'3', '4', '2', '6', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]
['1', '3', '4'] [LB: 13.959746629464261]

===== Iteration 166 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3', '4'] [LB: 13.959746629464261]

--> Optimal solution of ['1', '3', '4'] (Opt: 16.247737932508244): ['1', '3',
'4', '6', '2', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]
['1', '3'] [LB: 12.96192249819801]

===== Iteration 168 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874
Treat problem ['1', '3'] [LB: 12.96192249819801]

--> Optimal solution of ['1', '3'] (Opt: 16.247737932508244): ['1', '3', '4',
'6', '2', '5']

List of problems to treat:

['1'] [LB: 11.622630641882171]

===== Iteration 169 =====

Current best tour: ['1', '4', '2', '6', '3', '5'] Length: 14.080995677811874

Treat problem ['1'] [LB: 11.622630641882171]
--> Optimal solution of ['1'] (Opt: 14.080995677811874): ['1', '4', '2', '6',
'3', '5']
No more problem to treat.

```
[21]: print(f'Optimal tour: {fullProblem.optimalTour}')  
      print(f'Length of the optimal tour: {fullProblem.optimalValue}')
```

Optimal tour: ['1', '4', '2', '6', '3', '5']
Length of the optimal tour: 14.080995677811874

```
[22]: %matplotlib inline  
fig, ax = plt.subplots()  
ax.scatter(x, y)  
for i in range(n):  
    ax.annotate(names[i], (x[i], y[i]))  
for i in range(len(fullProblem.optimalTour) - 1):  
    j1 = names.index(fullProblem.optimalTour[i])  
    j2 = names.index(fullProblem.optimalTour[i + 1])  
    ax.plot([x[j1], x[j2]], [y[j1], y[j2]], 'ro-')  
j1 = names.index(fullProblem.optimalTour[-1])  
j2 = names.index(fullProblem.optimalTour[0])  
p = ax.plot([x[j1], x[j2]], [y[j1], y[j2]], 'ro-')
```

